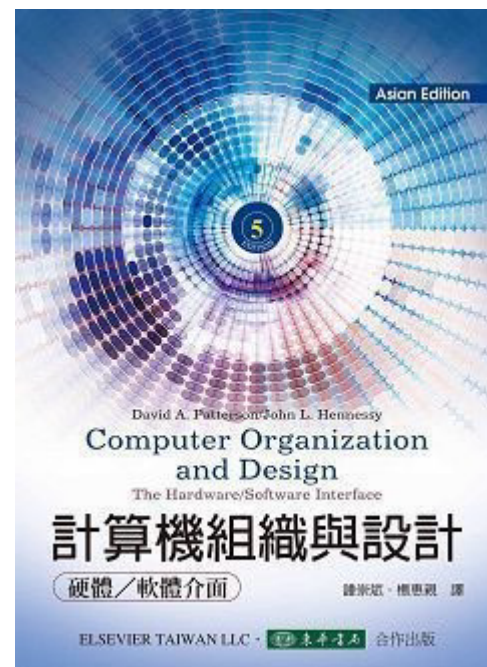
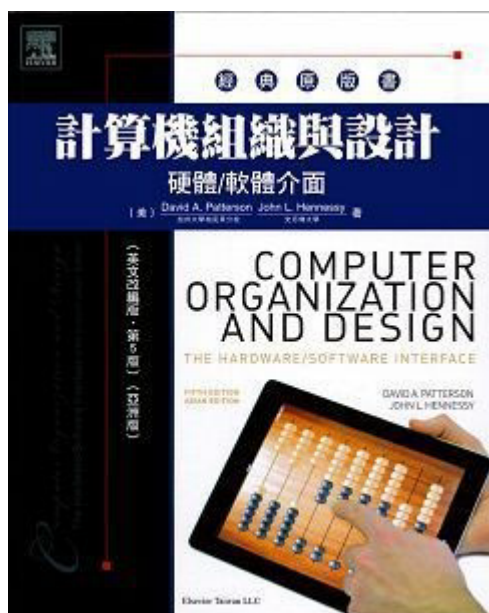


教科書



計算機組織與設計：硬體/軟體的介面 5/e
Patterson (Asian Edition)
作者：鍾崇斌、楊惠親 譯；David A.
Patterson

Computer Organization and Design 5/e
(Asian Edition)
作者：David A. Patterson, John L.
Hennessy

第 1 章

計算機抽象化與科技

1.1 介紹

1.2 計算機架構中的八大理念

1.3 你的程式之下

1.4 覆蓋之下

1.5 建構處理器與記憶體的技术

1.6 效能

1.7 功耗障壁

1.8 巨變：由單處理器轉移至多
處理器

1.9 實例：測試Intel core i7

1.10 謬誤與陷阱

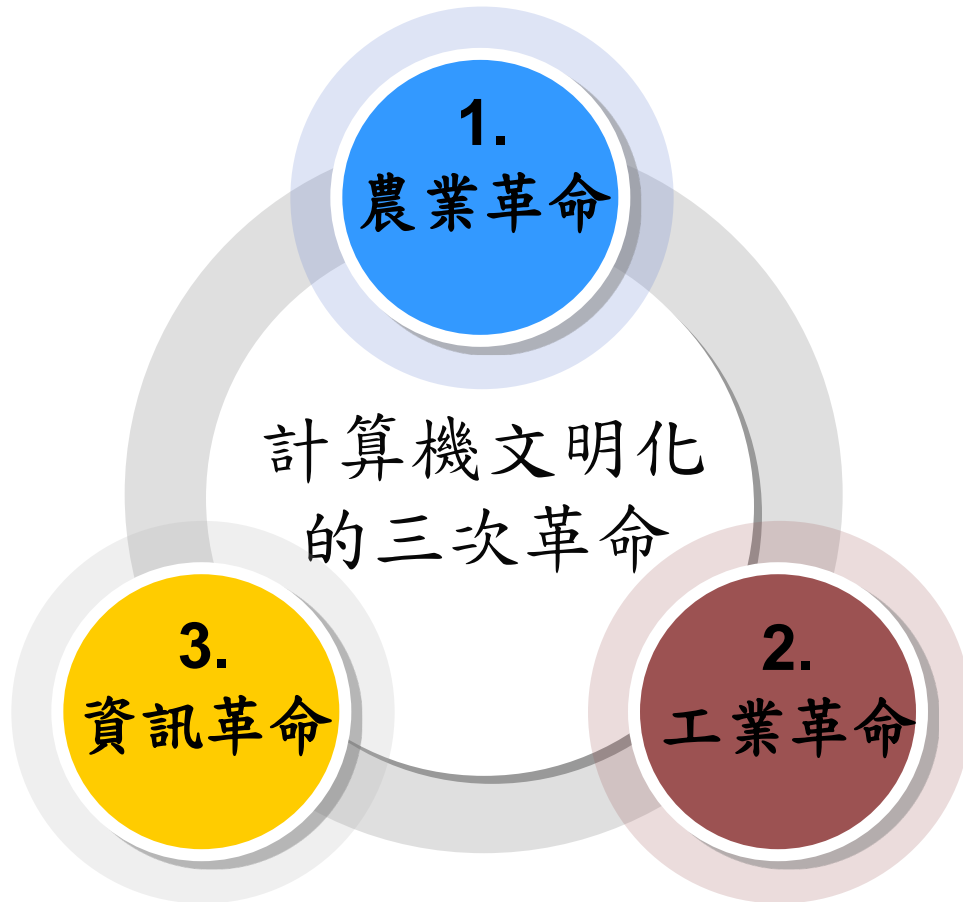
1.11 總結評論

1.12 歷史觀點與進一步閱讀

1.1 介紹

- ▣ 計算機是極為快速變化的資訊科技工業中的產物
 - 這個不尋常的工業以令人驚異的速度接納創意
 - 如果運輸工業也有計算機工業的進展速度，今天我們將大約可以花幾分錢以一秒鐘的時間由紐約去到倫敦

1.1 介紹



- 計算機在汽車中：1980
- 手機：1980
- 人類基因計畫：1990
- 全球網路：1990
- 搜尋引擎：1990

計算應用的種類與它們的特性

大體而言，計算機被運用在三個不同種類的應用：

▣ 個人型計算機(Personal computers, PCs)

- 強調以低成本提供單一使用者不錯的效能並能執行其他公司的軟體

▣ 伺服器(servers)是昔日大型主機、迷你計算機及超級電腦的現代版，通常經由網路來使用

- 提供計算及輸出入容量上更大的擴充性
- 也強調可靠度
- Range from small servers to building sized

計算應用的種類與它們的特性

- 嵌入式計算機(embedded computers)在應用與效能的範圍最廣
 - 設計來執行單一應用或一組相關的應用，該等應用一般與硬體整合在一起，以單一系統型態呈現
 - 大部分使用者從不知道他們正在使用這類計算機
 - 通常有其獨特的應用需求以及最低效能和嚴格的成本與功耗限制
 - 通常更不能容忍失效
 - 經由簡單化或冗餘(redundancy)技術來達成

進入後個人電腦時代

- ▣ 個人行動裝置(personal mobile device, PMD)由電池驅動、無線聯線至網際網路且一般售價為數百美元
 - 和個人電腦一樣，使用者可於其上執行可下載的軟體(“apps”)
 - 可能使用觸控螢幕甚或語言輸入
- ▣ 庫房規模計算機(Warehouse Scale Computers, WSCs)與雲端計算(Cloud Computing)
 - 軟體寫作者常常會將他們的應用程式一部分執行於個人行動裝置上而一部分執行於雲端

What is a WSC

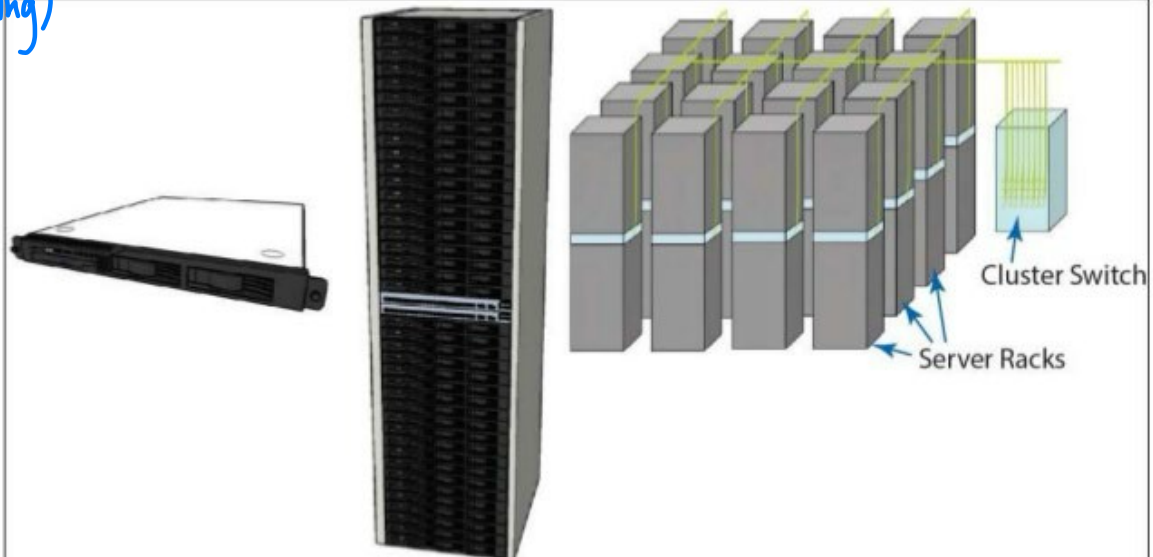


Warehouse-Scale Computer

- Scalable
- Distributed
- Cost efficiency

Server -----> WSC
 (Cloud Computing)
PCs -----> PMD
Embedded System -----> PMD

ARCHITECTURAL OVERVIEW OF WSCS



你可以在本書中學到

- ▣ 在1960及1970年代，計算機的效能主要受限於其記憶體的大小
- ▣ 現在重視效能的程式師需要瞭解，處理器的平行本質以及記憶體的階層性本質
 - 程式師於是必須增加他們的計算機組織知識
- ▣ 本書能回答以下問題：
 - 以高階語言如C 或Java 所寫的程式如何被翻譯成硬體的語言，以及硬體如何執行這個硬體語言的程式？

你可以在本書中學到

- 硬體與軟體的介面與關聯是什麼，以及軟體如何指使硬體來執行所需的功能？
- 什麼因素決定程式的效能，以及程式如何可改善其效能？
- 硬體設計師可用以改善效能的技術有哪些？
- 最近由循序處理轉為平行處理的理由以及結果是什麼？
- 自從第一台商用計算機在1951年面世以來，計算機架構師提出過哪些奠定現代計算基礎的大理念？

你可以在本書中學到

- ▣ 本書第一章奠下了本書其餘部分的基礎。
 - 介紹基本觀念與定義
 - 定位主要軟硬體元件的觀念
 - 說明如何評估效能與功耗
 - 介紹積體電路(其為促使計算機改革的驅動技術)
 - 解釋何以會走向多核心

瞭解程式效能

▣ 下表歸納了硬體及軟體如何影響效能

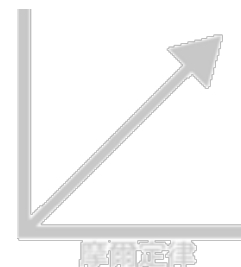
硬體或軟體元件	該元件如何影響效能	該主題見於何處？
演算法 →指令碼多少、CPI?	決定了原始碼行數及執行的輸入/輸出動作數	其他書本！
程式語言、編譯器與架構 ↑	決定原始碼中每一敘述句對應的計算機指令數目	第二及三章
處理器與記憶體系統 →指令集架構	決定指令可以被執行得多快	第四、五及六章
輸入/輸出系統(硬體與作業系統)	決定輸入/輸出動作可能被執行得多快	第四、五及六章

↪ 記憶體多大？多快？(SSD → DRAM → L3 → L2 → L1)
CPU 內

1.2 計算機架構中的八大理念

1. 配合摩爾定律作設計 Moore's Law

- 該定律指出IC 容量於每18 至24 個月即加倍



2. 用抽象化來簡化設計 Abstraction

- 一個硬體與軟體中的主要產能技巧就是如何恰當使用抽象化 (abstraction) 來代表在不同層次中的設計
- 在電腦科學中，抽象化是將資料與程式，以它的語意來呈現出它的外觀，但是隱藏起它的實作細節。一個電腦系統可以分割成幾個抽象層 (Abstraction layer)，使得程式設計師可以將它們分開處理。

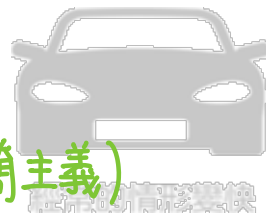


1.2 計算機架構中的八大理念

3. 使經常的情形變快 Make Common Case Fast

(讓 Clock 變快!)

\ RISC-Style / Minimalist (極簡主義)

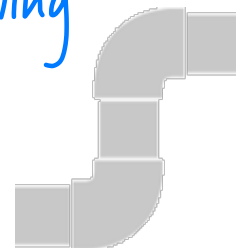


4. 經由平行性提升效能 Parallelism



平行性

5. 經由管道處理(pipelining)提升效能 Pipelining
(管線化)



管道化

1.2 計算機架構中的八大理念

6. 經由預測(prediction)提升效能 Prediction
分支預測

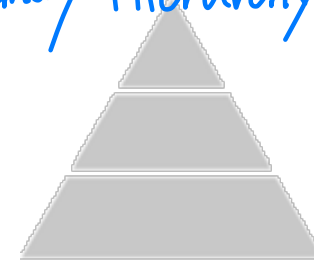


預測

7. 記憶體的階層(hierarchy of memories) Memory Hierarchy

- 程式設計師希望記憶體要容量大、速度快且價廉

容量大 ← 容量小
速度慢 ← 速度快
(SSD → DRAM → L3 → L2 → L1)
CPU 內



階層

8. 經由冗餘(redundancy)提升可靠性 Redundancy



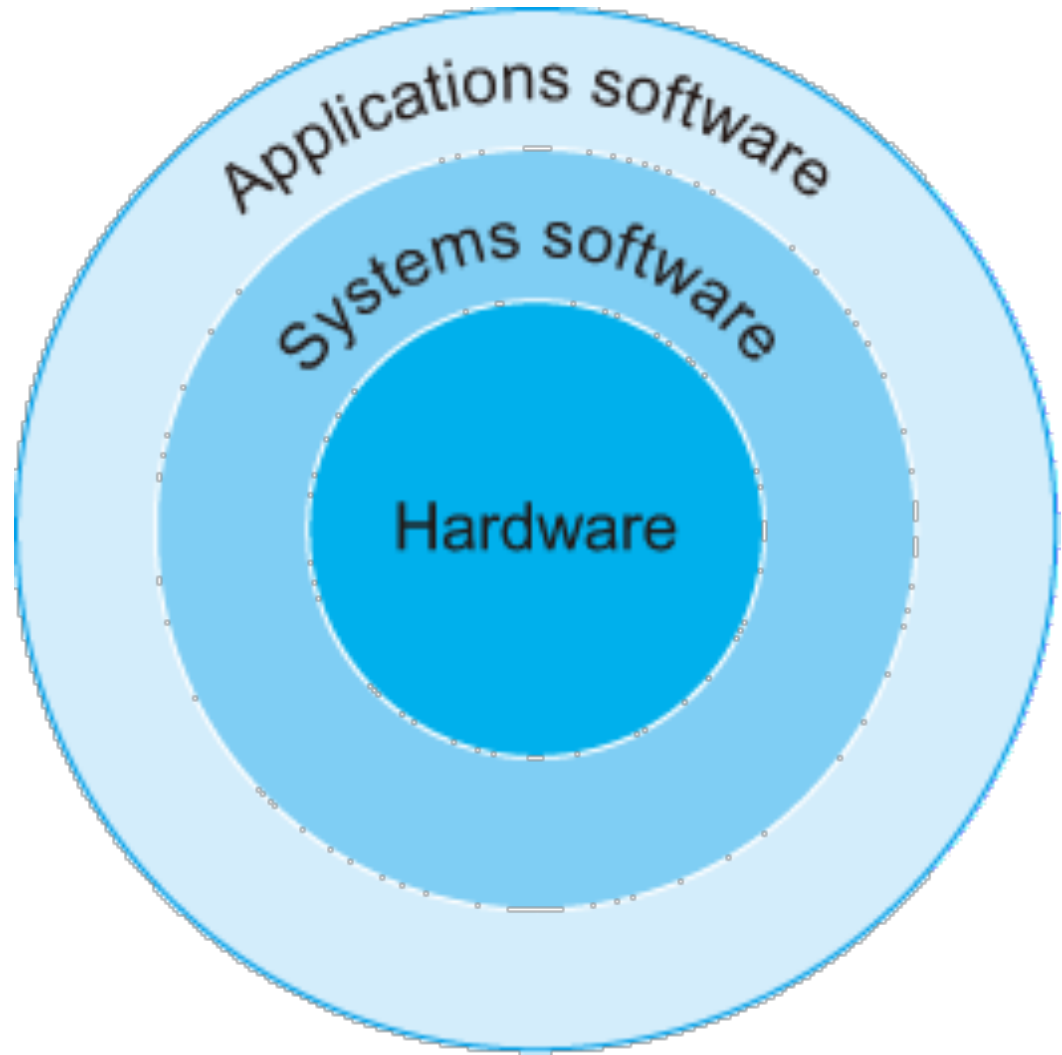
可靠性

1.3 你的程式之下(Below Your Program)

- ▣ 計算機中的硬體僅能執行極為簡單的低階指令
- ▣ 從複雜的應用落實到這些簡單硬體指令，過程中牽涉到許多層的軟體以直譯(interpret)或翻譯(translate)高階動作成為簡單計算機指令
編譯 Compiler
- ▣ 圖1.3 顯示該各層軟體被安排成階層式
- ▣ 系統軟體有許多種類，然而現在每一個計算機中最重要的有兩類：
 - ① 作業系統(operating system)
 - ② 編譯器

圖1.3 以硬體為中心而
應用軟體在最外的同心圓表示的簡化硬體
軟體階層圖

在複雜的應用情形下，
也常會用到多層的應
用軟體。



由高階語言到硬體的語言

- ▣ 計算機硬體的字母僅有兩個，為二進制數元(binary digit)或位元(bit)
- ▣ 計算機受我們的命令——稱之為指令(instruction) 所役使
- ▣ 組譯器(assembler)將一道以符號表示的指令翻譯成二進形式

add A,B $\Rightarrow$ 1000110010100000

- ▣ 高階程式語言(high-level programming languages)、組合語言(assembly language)與機器語言(machine language)，圖1.4 表示該等程式與語言間的關係

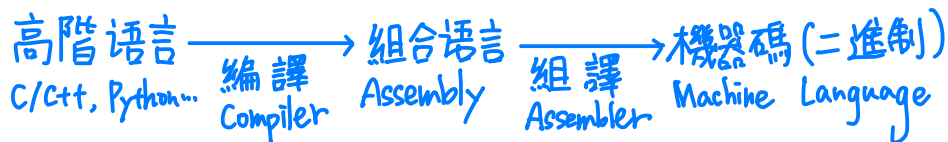


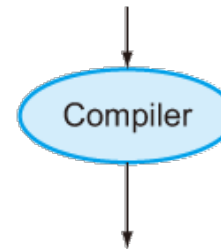
圖1.4 C 程式編譯成組合語言再組譯成二進制機器語言。

雖然由高階語言轉譯成二進制機器語言在此表示為兩個步驟，有些編譯器省去中間過程而直接產出二進制機器語言。這些語言以及這個程式將在第二章中有更深入的討論

High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

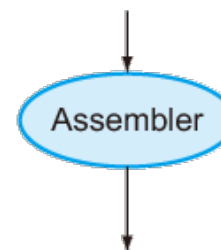
編譯器



Assembly
language
program
(for MIPS)

```
swap:
  multi $2, $5, 4
  add   $2, $4, $2
  lw    $15, 0($2)
  lw    $16, 4($2)
  sw    $16, 0($2)
  sw    $15, 4($2)
  jr    $31
```

組譯器



Binary machine
language
program
(for MIPS)

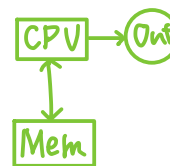
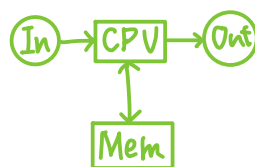
```
000000001010001000000000100011000
00000000100000100001000000100001
10001101111000100000000000000000
100011100001001000000000000000100
10101110000100100000000000000000
101011011110001000000000000000100
00000011111000000000000000001000
```

1.4 覆蓋之下(Under the Covers)

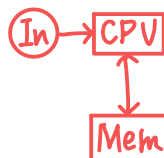
▣ 所有計算機內的硬體都執行以下的基本功能：

- I/O (① 輸入資料 (可以沒有) ② 輸出資料 (一定要有!))
- Process (③ 處理資料)
- Mem Disk (④ 儲存資料)

Accept



No Accept

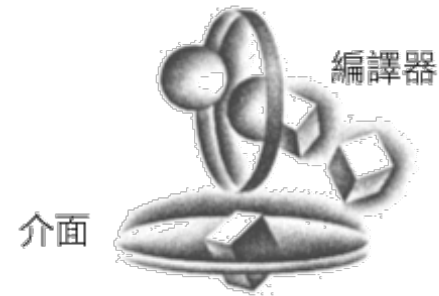


1.4 覆蓋之下

- ▶ 計算機的五大傳統標準要件是輸入、輸出、記憶體、數據通道(datapath)以及控制，而最後兩項有時會併稱為處理器。
CPU
- ▶ 圖1.5 顯示計算機的標準組織

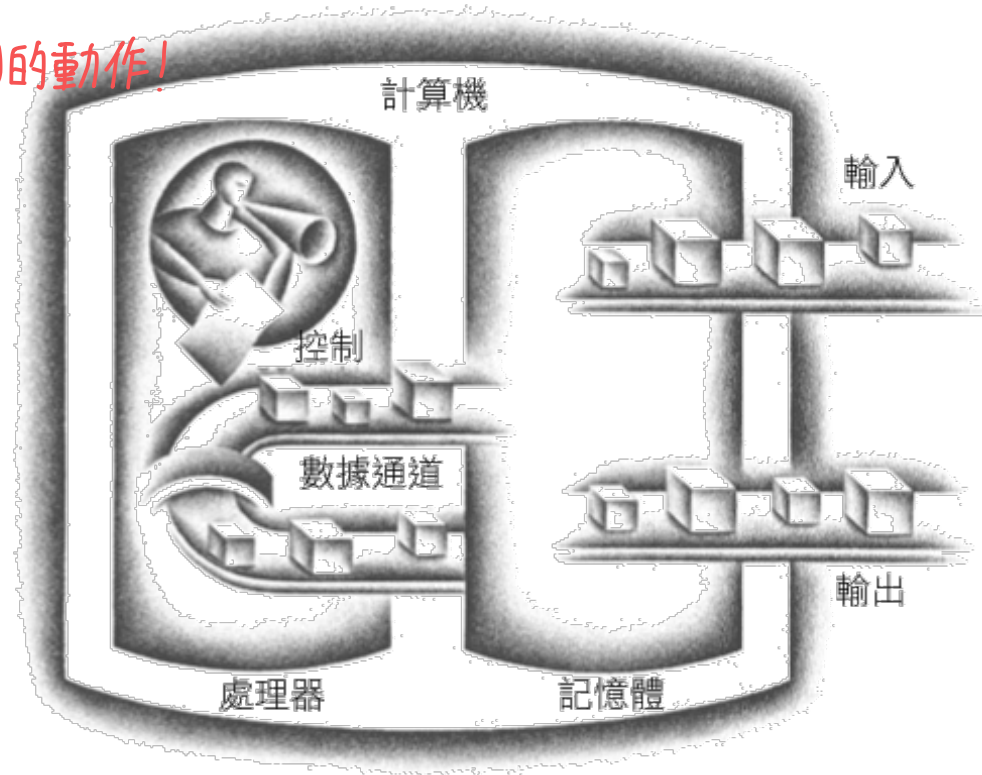
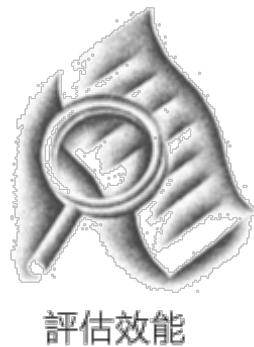
圖1.5 表示出五大標準要件的計算機組織

處理器由記憶體取得指令與資料。輸入將資料寫入記憶體，而輸出由記憶體讀取資料。控制送出決定數據通道、記憶體、輸入和輸出應如何運作的訊號



指令的目的：

讓 Data 在不同數據通道間移動，實現不同的動作！



1.5 建構處理器與記憶體的技术

Gates 數目
 SSI 1 ~ < 10
 MSI 10 ~ < 100
 LSI 100 ~ < 1000
 VLSI 1000 ~ < 10000
 ULSI 10000 ~

圖1.10 表示過去不同時期所使用的技術，並對每一種技術估計其單位成本的對應效能

年度	計算機使用的技術	相對的效能/單位成本
1951	真空管(vacuum tube) (電子管)	1
1965	電晶體 → 1947, 貝爾實驗室 應用① 通訊廣播領域: Amplifier 放大器 應用② 數位电路/计算机領域: Switch 開關	1st 利用电晶体 製造電腦 Inc: IBM 35
1975	積體電路	900
1995	超大型積體電路 VLSI	2,400,000
2013	極大型(Ultra large-scale)積體電路 ULSI	250,000,000,000

圖1.10 不同年代計算機中所使用技術的每單位成本的相對效能。

資料來源：Computer Museum，美國Boston，而2013 年的數據是由作者依外插的方式得出

1.5 建構處理器與記憶體的技术

■ 圖1.11 表示動態隨機記憶體容量自1977年以來的增長

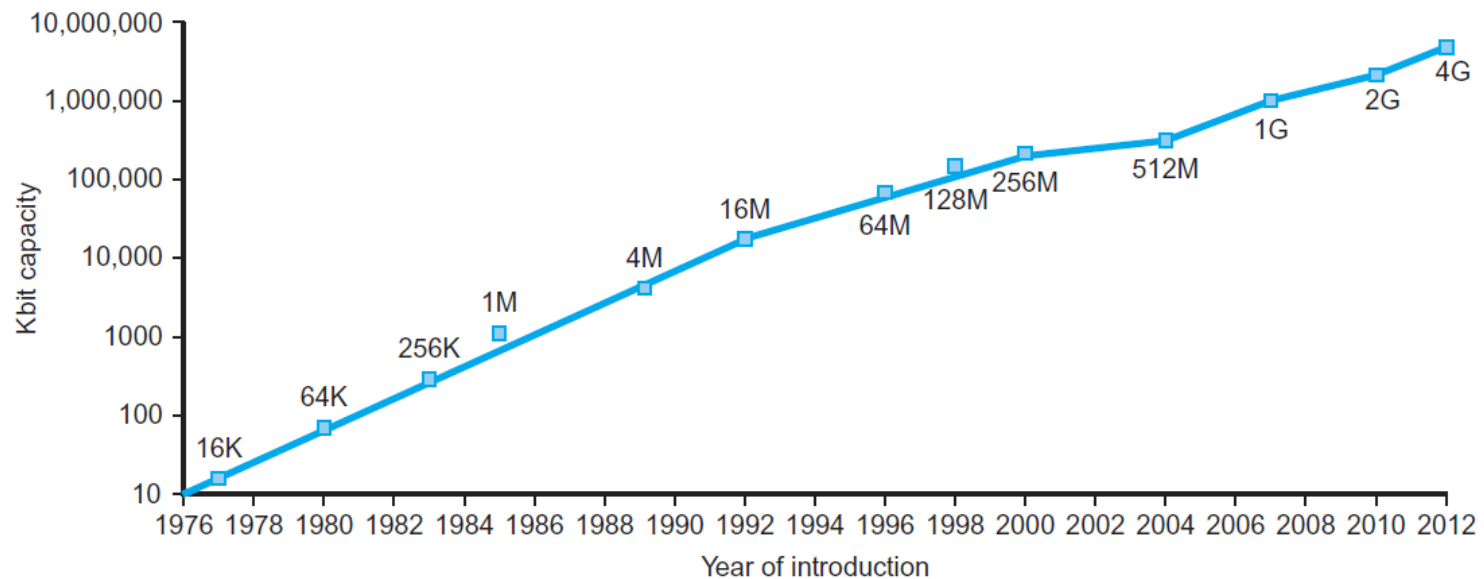


圖1.11 DRAM 晶片容量隨著時間的成長曲線

y 軸以kbits (2^{10} bits)作為單位。DRAM 業界在過去20年來幾乎每三年就將容量提高四倍，也就是每年提升60%。近年來，這個增加速度已經緩慢下來，漸漸變成約略每兩年到三年才將容量加倍

1.5 建構處理器與記憶體的技术

重要！

圖1.12 表示積體電路製程

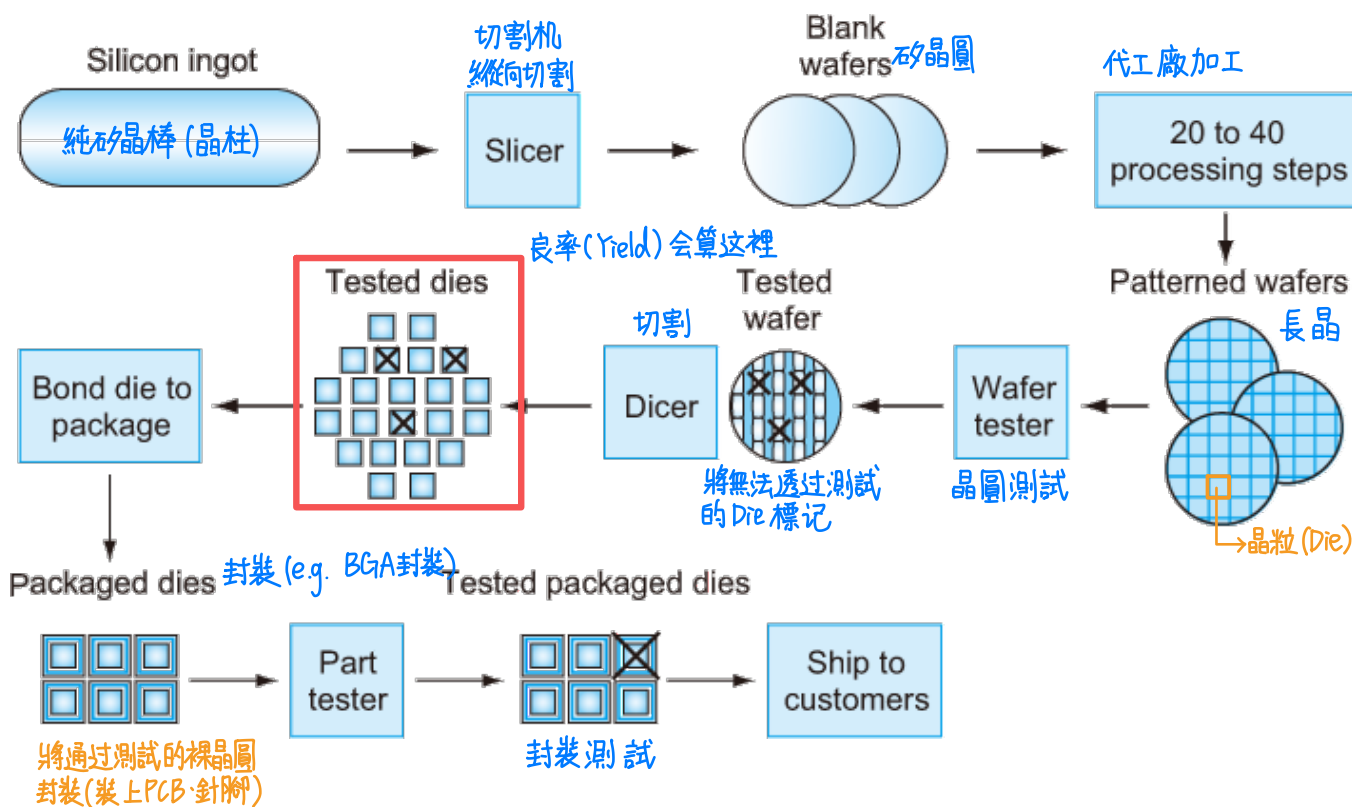
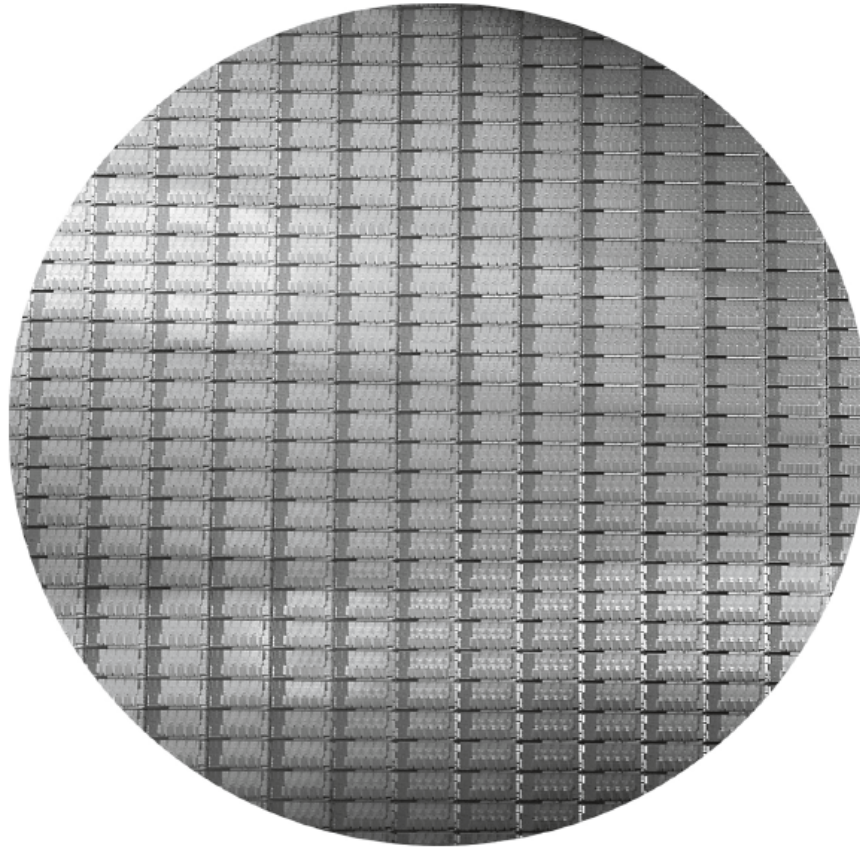


圖1.12 晶片的製造過程
空白晶圓從矽錠切片下來後，經過20到40道步驟後成為作好電路的晶圓(見圖1.13)。之後這些作好電路的晶圓經過晶圓測試機測試以產生良好部分的對照圖。之後晶圓被切割成晶粒(見圖1.9)。本圖中，一片晶圓產出20個晶粒，其中17個通過測試(x表示晶粒損壞)。本例中良好晶粒的良率(產出率，yield)是17/20或85%。這些好的晶粒接著被連結在封裝中並於出貨給顧客之前再作一次測試。在這個最後的測試中又發現一個壞掉的封裝好的零件

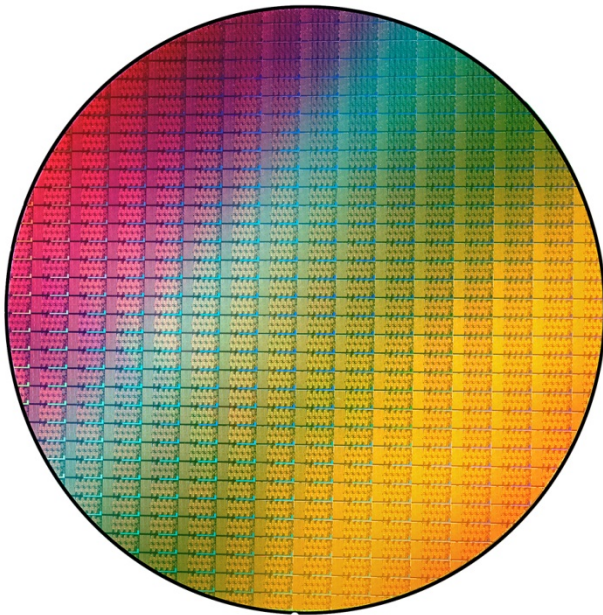
Intel Core i7 Wafer



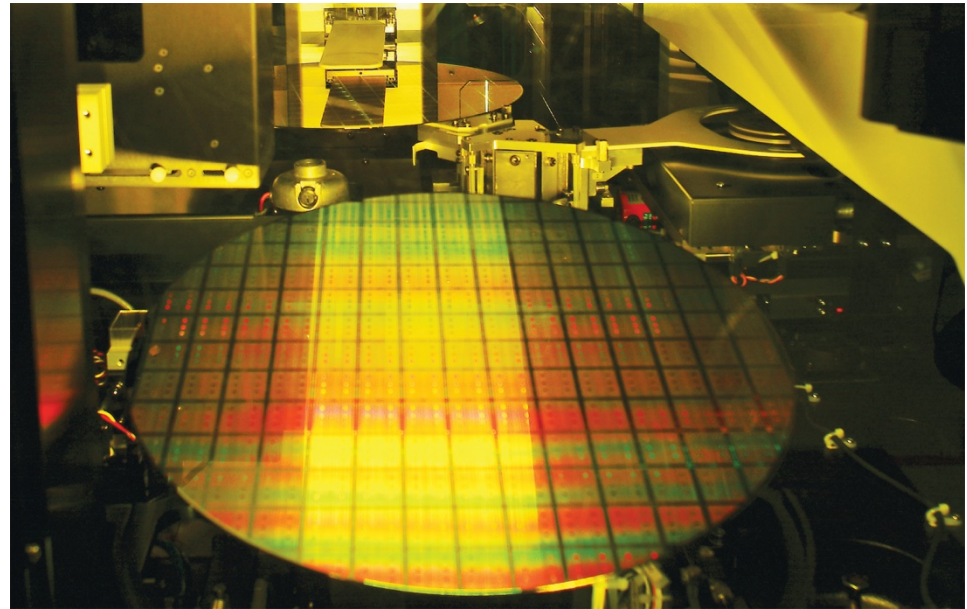
12 吋

- 300mm wafer, 280 chips, 32nm technology
- Each chip is 20.7 x 10.5 mm

Wafers: Intel Core i9 vs Apple Bionic A14



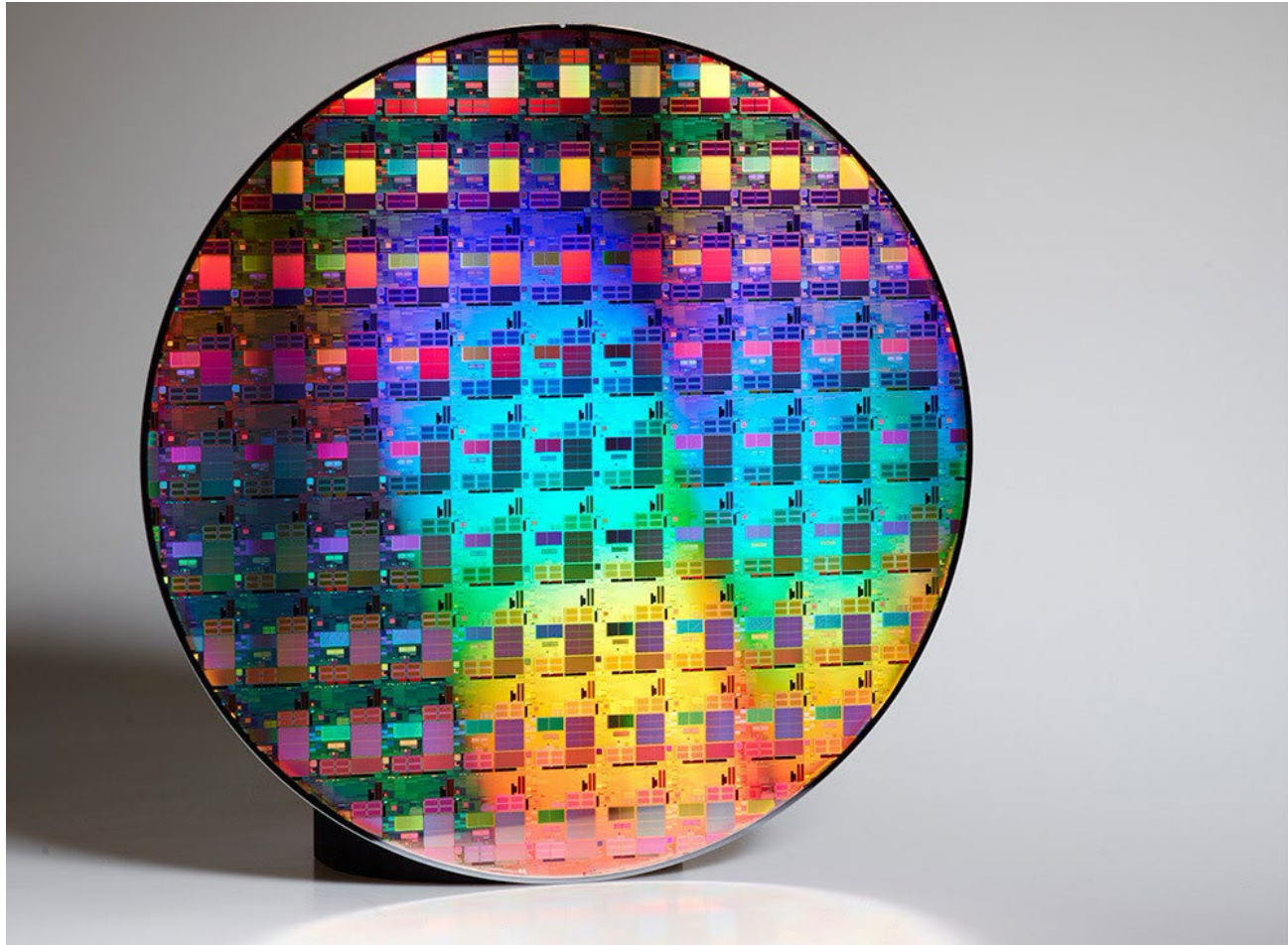
Intel i9 14nm CPU



Apple A14 TSMC 5nm CPU

■ 300mm wafer, Intel 14nm vs. TSMC 5nm

Wafer: Samsung Exynos Processor ?



- ▣ 300mm wafer, Samsung 3nm

Integrated Circuit Cost

晶圓成本

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{Yield}}$$

晶圓能切出幾個die 良率

$$\text{Dies per wafer} \approx \text{Wafer area} / \text{Die area}$$

$$\text{Yield} = \frac{1}{(1 + (\text{Defects per area} \times \text{Die area} / 2))^2}$$

單位面積瑕疵比例 die 的面積

單位面積瑕疵越高 $\Rightarrow$ 良率越低
單位面積越大

<http://bnrg.eecs.berkeley.edu/~randy/Courses/CS252.S96/Lecture05.pdf>

■ Nonlinear relation to area and defect rate

- Wafer cost and area are fixed
- Defect rate determined by manufacturing process
- Die area determined by architecture and circuit design

"晶圓面積"由架構 & 电路设计決定

$\Rightarrow$ 晶圓面積 & 瑕疵率 決定良率

1.6 效能

$$\text{R.T.} = \text{E.T.} = \text{Latency (延遲)}$$

✓ (Response Time) (Execution Time)

▣ Response time(反應時間) 越小越好

- How long it takes to do a task (作一個工作要花多久)

▣ Throughput(處理量、吞吐量) 越大越好

- Total work done per unit time (單位時間完成的總工作量)
 - e.g., tasks/transactions/... per hour

▣ 個人關心的是降低response time——一件工作由開始至結束的時間，亦稱為執行時間(execution time)

▣ 數據中心管理員則常關心處理量(throughput)或頻寬(bandwidth)在給定時間內所完成的工作量

處理量與反應時間

以下對計算機系統的改變可否提昇處理量、降低反應時間或兼得？

1. 以較快的處理器置換於計算機中 1-2 PPT P.14
「使經常的情形變快」 Response Time ↑
Throughput 可能也 ↑
2. 在使用多個處理器來分別處理各個工作——例如在全球資訊網中搜尋——的系統中加入額外的處理器 → Response Time 持平, 但 Throughput ↑

	原CPU	升級CPU
CPU數 不變	T.P. - L.A. -	T.P. - L.A. ↓
CPU數 增加	T.P. ↑ L.A. -	T.P. ↑ L.A. ↓

處理量與反應時間

- 降低反應時間幾乎永遠可提升處理量。因此在情況1中，反應時間及處理量均獲改善。在情況2中，沒有任何工作可更快完成，故僅處理量有提昇
- 唯當情況2中對處理的需求大到幾乎與處理量相同時，可能造成系統將工作需求貯存起來。在這種情況下，由於增加處理量可降低工作需求在貯列中的等待時間，故可同時改善反應時間。因此，在許多實際的計算機系統中，改變執行時間或處理量通常會影響另一者

Relative Performance(相對效能)

- ▣ Define Performance = 1/Execution Time(相對效能是執行時間的倒數)
- ▣ “X is n time faster than Y”

$$\begin{aligned} & \text{Performance}_X / \text{Performance}_Y \\ &= \text{Execution time}_Y / \text{Execution time}_X = n \end{aligned}$$

- **Example: time taken to run a program**
 - 10s on A, 15s on B
 - $\text{Execution Time}_B / \text{Execution Time}_A$
 $= 15\text{s} / 10\text{s} = 1.5$
 - So A is 1.5 times faster than B

Measuring Execution Time(測量執行時間)

■ Elapsed time (所有經過的時間)

- Total response time, including all aspects (含各方面全部反應時間)
 - Processing, I/O, OS overhead, idle time
CPU 處理時間 OS、系統程式浪費的時間 待機時間
- Determines system performance (決定系統效能)

■ CPU time (CPU時間) → CPU 時間: CPU單純服務你的時間(扣除 I/O、Idle Time.....)

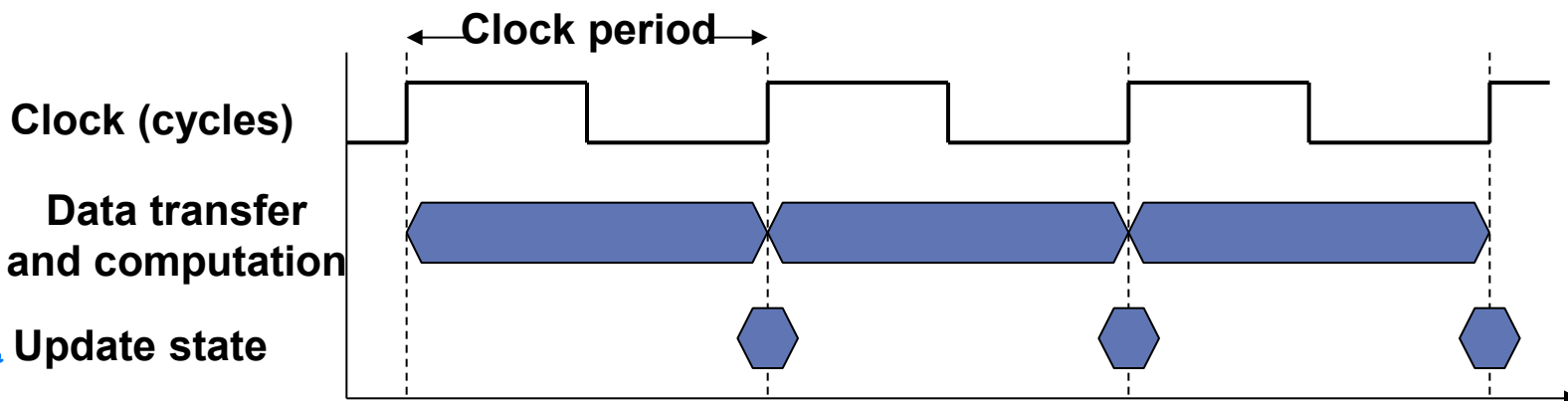
- Time spent processing a given job(花在處理特定工作的時間)
 - Discounts I/O time, other jobs' shares (不計 I/O)
- Comprises user CPU time and system CPU time
CPU 服務你的程式的時間 你的程式呼叫的 system call 所耗的時間
- Different programs are affected differently by CPU and system performance (不同程式受CPU與系統效能的影響不同) → 不同程式会被不同的CPU & 系統效能所影響!

CPU Clocking (時脈週期與頻率互為倒數)

- Operation of digital hardware governed by a constant-rate clock

一个Clock, 一个核心
只做一个指令

$$\frac{1}{250(\text{ps})} = \frac{1}{250 \times 10^{-12}} = \frac{4}{25} \times 10^9 = 4 \times 10^9 \text{ (Hz)} = 4000 \text{ MHz} = 4.0 \text{ GHz}$$



- Clock period: duration of a clock cycle**

- e.g., $250\text{ps} = 0.25\text{ns} = 250 \times 10^{-12}\text{s}$

- Clock frequency (rate): cycles per second**

- e.g., $4.0\text{GHz} = 4000\text{MHz} = 4.0 \times 10^9\text{Hz}$

P 10^{-12}
n 10^{-9}
μ 10^{-6}
m 10^{-3}
K 10^3
M 10^6
G 10^9
T 10^{12}

CPU Time

$$\begin{aligned}\text{CPU Time} &= \text{CPU Clock Cycles} \times \text{Clock Cycle Time} \\ &= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}\end{aligned}$$

周期 (= CPU 频率倒数)

- ▣ Performance improved by 改善效能 (CPU Time ↓)
 - Reducing number of clock cycles 減少 Clock Cycles 數
 - Increasing clock rate 增加 CPU 频率
 - Hardware designer must often trade off clock rate against cycle count

CPU 要跑得快 → 指令就不能太複雜 ⇒ 硬體設計得在 Clock Rate & 指令數目
做一個妥協

中央處理器效能及其因素

$$\text{中央處理器執行某程式需時} = \frac{\text{中央處理器執行該程式所需時脈數}}{\text{時脈週期時間}} \times \text{時脈週期時間}$$

$$\text{中央處理器執行某程式需時} = \frac{\text{中央處理器執行該程式所需時脈數}}{\text{時脈速率}}$$



指令效能

▣ 上述的效能計算式並未參考到執行程式所需的指令數。

▣ 中央處理器時脈數

= 程式所需指令數 × 指令平均時脈數

■ 每指令時脈數(clock cycles per instruction)，通常簡稱為**CPI** (一个平均值)



典型中央處理器效能計算式

■ 中央處理器時間 = $\frac{\text{指令數} \times \text{CPI} \times \text{時脈週期}}{\text{時脈速率}}$

|| $\frac{\text{Clock Cycles}}{\text{時脈速率}}$

中央處理器時間 = $\frac{\text{指令數} \times \text{CPI}}{\text{時脈速率}}$

- 唯一完整而可靠的計算機效能量測就是時間
- 時脈週期通常會包含於一個計算機印行的文件中
- 於指令數僅與架構有關而與確實的製作無關
- CPI 與許多不同的計算機設計細節有關，包括記憶體系統與處理器結構兩方面以及應用程式中所執行的指令類別混合比



Practice

每个 Command 要几个 Cycle 才能做完?

— CPI越低越好

指令數目

Clock Cycles

Clock Rate

- 產生指令數目？

- Architecture) and compiler 指令集架構

- 每個指令平均 Cycle 數由硬體決定

- Average CPI affected by instruction mix

瞭解程式效能

$$\text{CPU Time} = \frac{\text{指令數}}{\text{Program}} \times \frac{\text{Clock Cycles}}{\text{Instruction}} \times \frac{\text{週期}}{\text{Clock Cycle}}$$

指令數
 一個程式有多少指令？
 CPI
 一個指令有多少Cycle？
 週期
 每個Cycle所需的時間

一個程式的效能與演算法、語言、編譯器、架構以及實際的硬體有關

硬體或軟體要素	影響什麼？	如何影響？
演算法	指令數，可能 CPI	演算法決定執行的高階程式指令數目，也因此決定執行的處理器指令數目。演算法對較快或較慢指令的偏好也可能影響 CPI。舉例而言，若演算法使用較多的浮點運算，則可能導致更高的 CPI。
程式語言	指令數，CPI	程式語言當然影響指令數，因為其敘述句將被轉譯成處理器指令，因而決定指令數。語言的特性也可能影響 CPI；舉例而言，一個大量支援資料抽象化(data abstraction)的語言(例如 Java)將需要間接呼叫的功能，如此將使用 CPI 較高的指令。
編譯器	指令數，CPI	由於編譯器決定將高階語言指令轉換成計算機的指令，其效率會影響指令數及每指令平均週期數。編譯器的角色可以相當複雜，並以複雜的方式影響 CPI。
指令集架構	指令數，時脈速率，CPI	由於指令集架構影響一個功能所需的指令、每道指令的時脈數以及處理器整體時脈速率，因此其影響中央處理器效能相關的所有方面。



仔細深思

- 有些處理器每時脈週期擷取及執行多道指令
 - ➡ 有些設計者倒轉CPI以表為IPC，或每時脈指令數。
 - ➡ 若某處理器平均每時脈執行二道指令，則其具有 $IPC=2$ 亦即 $CPI=0.5$
- 為了省能或是短暫地加強效能，現在的許多處理器可以變化它們的時脈速率，因此我們對一個程式會需要知道其平均的時脈速率



CPU Time Example

A 總共執行了 $2 \times 10^9 \times 10$ 個 CPU cycles

Computer A: 2GHz clock, 10s CPU time

(1sec : 2×10^9)

Designing Computer B

- Aim for 6s CPU time

- Can do faster clock, but causes $1.2 \times$ clock cycles

How fast must Computer B clock be?

問頻率!

$$\text{Clock Rate}_B = \frac{\text{Clock Cycles}_B}{\text{CPU Time}_B} = \frac{1.2 \times \text{Clock Cycles}_A}{6s}$$

$$\begin{aligned}\text{Clock Cycles}_A &= \text{CPU Time}_A \times \text{Clock Rate}_A \\ &= 10s \times 2\text{GHz} = 20 \times 10^9\end{aligned}$$

$$\text{Clock Rate}_B = \frac{1.2 \times 20 \times 10^9}{6s} = \frac{24 \times 10^9}{6s} = 4\text{GHz}$$

P 10^{-12}
n 10^{-9}
μ 10^{-6}
m 10^{-3}
K 10^3
M 10^6
G 10^9
T 10^{12}

CPI Example

* Cycles Time 就是 "週期" (CPU 頻率倒數)

也就是每個週期所需耗的時間 $\Rightarrow$ 越小越好!

- Computer A: Cycle Time = 250ps, CPI = 2.0
- Computer B: Cycle Time = 500ps, CPI = 1.2
- Same Instruction count
- Which is faster, and by how much?

$$\text{CPU Time}_A = \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A$$

$$= 1 \times 2.0 \times 250\text{ps} = 1 \times 500\text{ps}$$

A is faster...

$$\text{CPU Time}_B = \text{Instruction Count} \times \text{CPI}_B \times \text{Cycle Time}_B$$

$$= 1 \times 1.2 \times 500\text{ps} = 1 \times 600\text{ps}$$

$$\frac{\text{CPU Time}_B}{\text{CPU Time}_A} = \frac{1 \times 600\text{ps}}{1 \times 500\text{ps}} = 1.2$$

$\hookrightarrow$ A 比 B 快了 1.2 倍!

...by this
much

P 10^{-12}
n 10^{-9}
 μ 10^{-6}
m 10^{-3}
K 10^3
M 10^6
G 10^9
T 10^{12}

CPI in More Detail

- If different instruction classes take different numbers of cycles

當不同指令有不同CPI時……

Clock Cycles 就是加權平均數！

*n*種不同的指令

$$\text{Clock Cycles} = \sum_{i=1}^n (\text{CPI}_i \times \text{Instruction Count}_i)$$

■ Weighted average CPI

$$\text{CPI} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \sum_{i=1}^n \left(\text{CPI}_i \times \frac{\text{Instruction Count}_i}{\text{Instruction Count}} \right)$$

Relative frequency

CPI Example

- Alternative compiled code sequences using instructions in classes A, B, C

- IC : Instruction count (指令数)

Class	A	B	C
CPI for class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

Different
Compiler

- Sequence 1: IC = 5
(2+1+1)

- Clock Cycles
 $= 2 \times 1 + 1 \times 2 + 2 \times 3$
 $= 10$

- Avg. CPI = $10/5 = 2.0$

- Sequence 2: IC = 6
(4+1+1)

- Clock Cycles
 $= 4 \times 1 + 1 \times 2 + 1 \times 3$
 $= 9$

- Avg. CPI = $9/6 = 1.5$

Performance Summary

The BIG Picture

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

- ▣ Performance depends on
 - Algorithm: affects IC, possibly CPI
 - Programming language: affects IC, CPI
 - Compiler: affects IC, CPI
 - Instruction set architecture: affects IC, CPI, T_c

瞭解程式效能

- 一個程式的效能與演算法、語言、編譯器、架構以及實際的硬體有關

硬體或軟體要素	影響什麼？	如何影響？
演算法	指令數，可能 CPI	演算法決定執行的高階程式指令數目，也因此決定執行的處理器指令數目。演算法對較快或較慢指令的偏好也可能影響 CPI。舉例而言，若演算法使用較多的浮點運算，則可能導致更高的 CPI。
程式語言	指令數，CPI	程式語言當然影響指令數，因為其敘述句將被轉譯成處理器指令，因而決定指令數。語言的特性也可能影響 CPI；舉例而言，一個大量支援資料抽象化(data abstraction)的語言(例如 Java)將需要間接呼叫的功能，如此將使用 CPI 較高的指令。
編譯器	指令數，CPI	由於編譯器決定將高階語言指令轉換成計算機的指令，其效率會影響指令數及每指令平均週期數。編譯器的角色可以相當複雜，並以複雜的方式影響 CPI。
指令集架構	指令數，時脈速率，CPI	由於指令集架構影響一個功能所需的指令、每道指令的時脈數以及處理器整體時脈速率，因此其影響中央處理器效能相關的所有方面。

- 有些處理器每時脈週期擷取及執行多道指令
- ➡ 有些設計者倒轉CPI以表為IPC，或每時脈指令數。


IPC、CPI互為倒數

Instruction Per Clock
每个 Clock 能產生多少指令
- ➡ 若某處理器平均每時脈執行二道指令，則其具有 $IPC=2$ 亦即 $CPI=0.5$
- 為了省能或是短暫地加強效能，現在的許多處理器可以變化它們的時脈速率，因此我們對一個程式會需要知道其平均的時脈速率

1.7 功耗障壁

- ❑ 功耗形成了冷卻能力的最低要求
- ❑ 對能源而言焦耳這個能量單位比譬如瓦數的耗能率更為恰當
 - 所消耗的電能，稱為電功率，其單位為瓦特（watt，簡寫為W），簡稱瓦。◎如果有1安培的電流通過1伏特的電位差，則每秒內所獲得或消耗的能量為1焦耳，或者說電功率為1瓦特，即「1瓦特=1焦耳/秒」，或「1W=1J/s」。

IC基本上只會在轉態(0↔1)時產生功耗

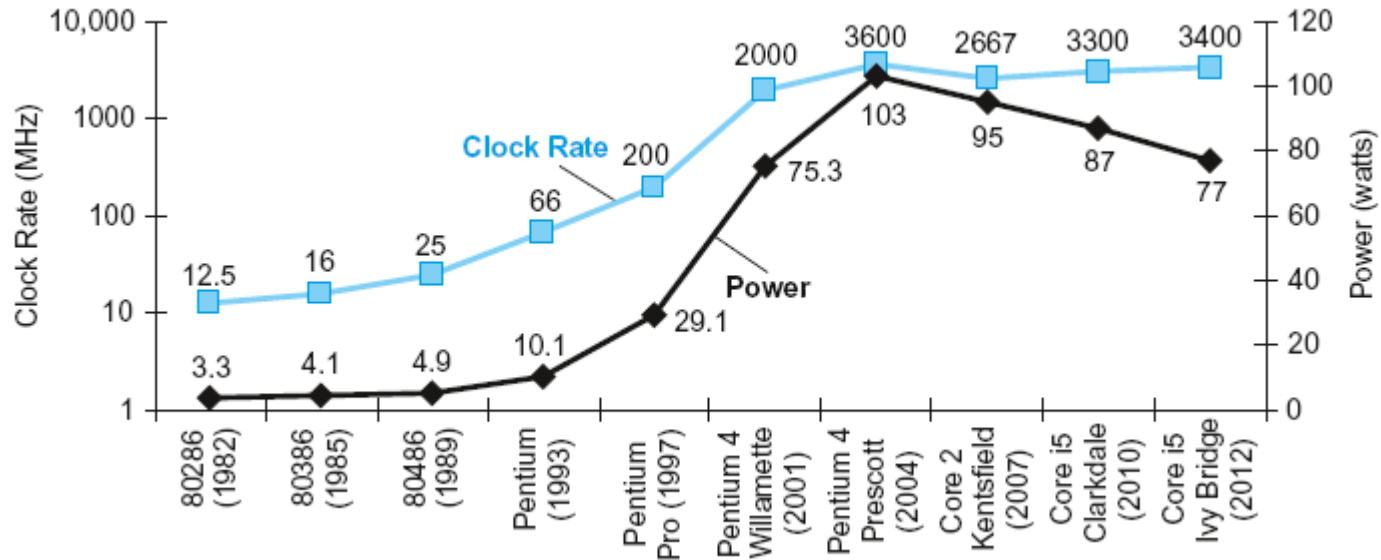
- ❑ 每個電晶體 0/1 轉換一次的動態能耗是  省电模式: 拉大 Clock, 減少轉態

$$\text{能耗} \propto \frac{1}{2} \times \text{電容性負載} \times \text{電壓}^2 \quad \left(\frac{1}{2} * C * V^2 \right)$$

- ❑ 每個電晶體所需功耗就是

$$\text{能耗} \propto \frac{1}{2} \times \text{電容性負載} \times \text{電壓}^2 \times \text{切換頻率} \quad \left(\frac{1}{2} * C * V^2 * f \right)$$

Power Trends



■ In CMOS IC technology

$$\text{Power} = \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency}$$

电容器負載

電壓²

頻率

×30

5V → 1V

×1000

功耗障壁

- ▣ 一般電壓在每個新世代中下降大約15%
 - 20 年來，電壓已從5V 降到1V
- ▣ 今日的問題是更加降低電壓將會使得電晶體漏電太多
 - 甚至於今天電耗中已有40% 來自於漏電
- ▣ 兩個理由使功耗成為積體電路設計的難題
 - ① 電源 必須由外部引入且分送至晶片各處 → 電怎麼進去？
 - ② 功耗以熱的形式消散然後必須被移除 → 熱怎麼出來？

1.8 巨變：由單處理器轉移至多處理器

- 圖1.17 顯示桌上型微處理器在程式反應時間上隨著年代的改善
 - 2002 年起改善速率減緩
- 2006 年時所有桌上型及伺服器公司都推出每一晶片具有多個處理器(多核心)的微處理器
 - 其所帶來在處理量上的好處更甚於反應時間，稱處理器為內核
 - 稱這種微處理器為多核(心)微處理器(multicore microprocessors)

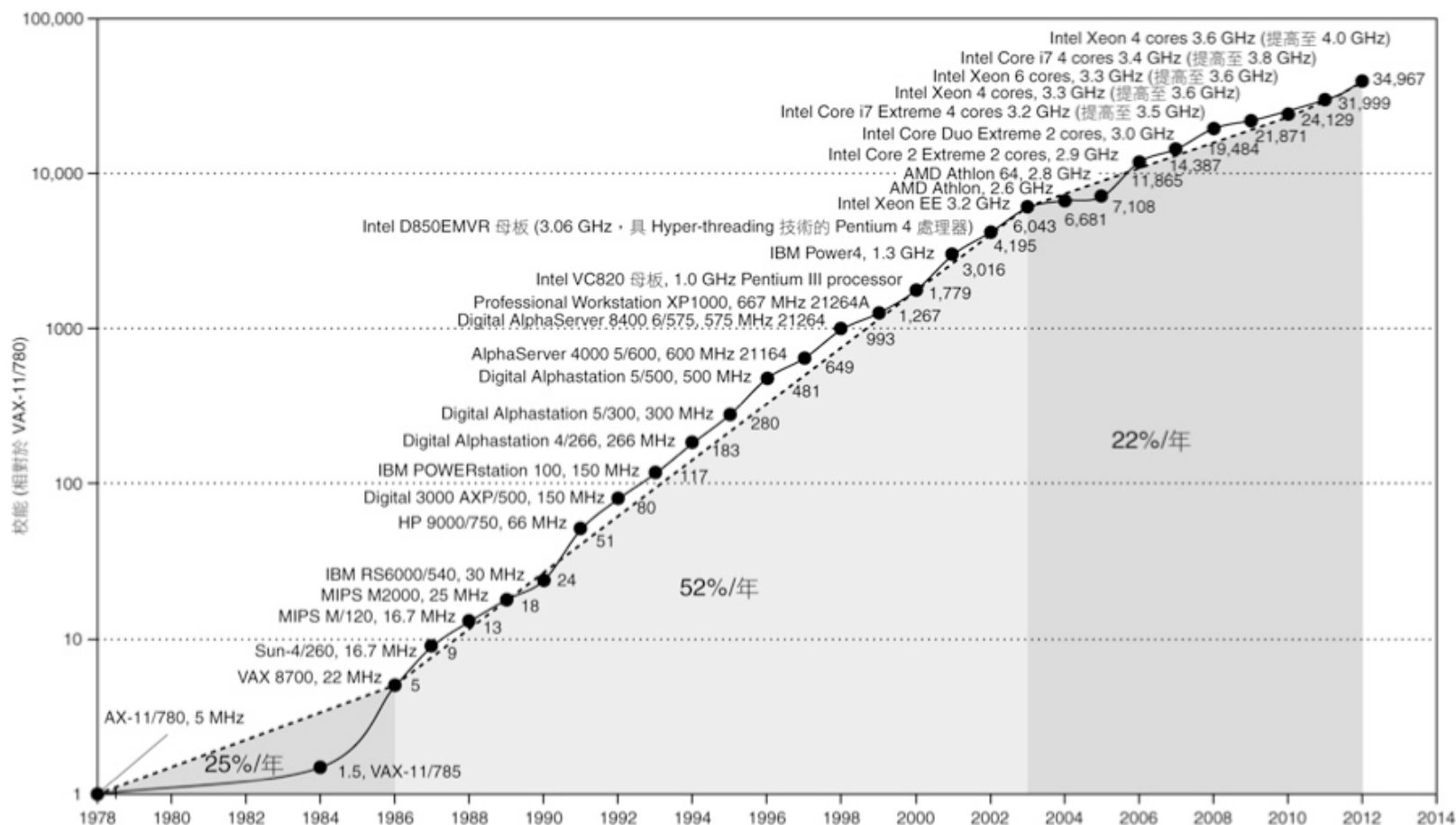


圖1.17 自1980年代中期以來的處理器效能增進情形

這個表描繪出以SPECint 測試程式(見1.10 節)測得的相對於VAX 11/780 的效能。在1980年代中期以前，處理器效能增進的動力主要是來自半導體技術，並且平均每年提升約25%。之後增進的速度提高到大約52%則是受惠於更為進步的架構與組織的觀念。這個自1980年代中期以來更高的52%效能增進表示：較之如果維持了每年25%的效能增進的話，到了2002年效能又多高出了七倍。自從2002年以來，功耗造成的各種限制、可用的指令階層平行度以及相對很長的記憶體延遲減緩了邇來的單一處理器效能增進，到了約略每年祇有22%的增進

1.8 巨變：由單處理器轉移至多處理器

- ▣ 今天，程式師如欲獲致重大的時間改善，則需要重新編程以善用多個處理器
- ▣ 為什麼寫作明確的平行程式這麼難？
 - 第一個理由是追求效能的程式寫作其困難度增加
 - 第二個理由是程式師必須分割一個應用以使每個處理器同時可以擁有大約相同的工作量，以及排程和協調所造成的額外花費不致於抵銷掉平行性帶來的潛在效能好處
 - 均勻地平衡負載(balance the load)
 - 降低通訊與同步的額外花費(reduce communication and synchronization overhead)

1.8 巨變：由單處理器轉移至多處理器

- 為了反映這個工業的巨變，以下五章均包含了一個有關平行革命對於該章涵意的小節：
 - 第二章，2.10 節：平行性與指令：同步
 - 第三章，3.6 節：平行性與計算機算術：部分字的平行性
 - 第四章，4.10 節：經由指令的平行性
 - 第五章，5.10 節：平行性與記憶體階層(memory hierarchy)：快取一致性
 - 第五章，5.11 節：平行性與記憶體階層：冗餘廉價磁碟陣列

1.8 巨變：由單處理器轉移至多處理器

- 另有一整章討論平行處理。第六章深入討論平行程式的挑戰
- ▣ 第三至六章透過矩陣向量相乘的實例來說明各種類型的平行性如何可以大大提升效能

1.9 實例：測試Intel Core i7

- 每章均有一個名稱是「實例」，用以連結書中的觀念以及你每日所使用的計算機的小節
- 這些小節述及現代計算機所使用的技術

SPEC 中央處理器測試程式

- 大多數使用者依賴已知方法來測量受測計算機的效能，並期待該方法足以反映某一計算機對他們的工作負載表現有多好

— e.g. 整數運算、浮點數運算……

- 通常是以一組測試程式(benchmarks)——特別挑選來量測效能的程式——來評量計算機
- 測試程式集組成一個使用者希望足以預測真實工作負載效能表現的工作負載

SPEC 中央處理器測試程式

- SPEC(系統效能評估組織，System Performance Evaluation Cooperative)是許多計算機業者為了創造現代計算機系統各種標準測試程式集而建立及支持的組織
 - 1989 年SPEC 首次提出SPEC89
 - 最新的一代稱為SPEC CPU2006
 - 包含了一組12 個的整數測試程式(CINT2006)以及17 個的浮點測試程式(CFP2006)

SPEC 中央處理器測試程式

▣ SPEC CPU2006

- Elapsed time to execute a selection of programs
 - Negligible(忽略) I/O, so focuses on CPU performance
- Normalize^{正規化} relative to reference machine
 - *SPECratio*
將測試資料正規化, 得出一個相對於參考機器的比值
- Summarize as geometric mean of performance ratios
 - CINT2006 (integer) and CFP2006 (floating-point)

$$\sqrt[n]{\prod_{i=1}^n \text{Execution time ratio}_i} \quad (\text{幾何平均數})$$

CINT2006 for Intel Core i7 920

Description	Name	Instruction Count x 10 ⁹	CPI	Clock cycle time (seconds x 10 ⁻⁹)	Execution Time (seconds)	Reference Time (seconds)	SPECratio
Interpreted string processing	perl	2252	0.60	0.376	508	9770	19.2
Block-sorting compression	bzip2	2390	0.70	0.376	629	9650	15.4
GNU C compiler	gcc	794	1.20	0.376	358	8050	22.5
Combinatorial optimization	mcf	221	2.66	0.376	221	9120	41.2
Go game (AI)	go	1274	1.10	0.376	527	10490	19.9
Search gene sequence	hmmer	2616	0.60	0.376	590	9330	15.8
Chess game (AI)	sjeng	1948	0.80	0.376	586	12100	20.7
Quantum computer simulation	libquantum	659	0.44	0.376	109	20720	190.0
Video compression	h264avc	3793	0.50	0.376	713	22130	31.0
Discrete event simulation library	omnetpp	367	2.10	0.376	290	6250	21.5
Games/path finding	astar	1250	1.00	0.376	470	7020	14.9
XML parsing	xalancbmk	1045	0.70	0.376	275	6900	25.1
Geometric mean	–	–	–	–	–	–	25.7

SPEC 功耗測試程式

■ SPEC 測試功耗的程式

■ SPECpower 測出伺服器在一段時間裡以10% 為區間的負載情況下的功耗

- 為了簡化計算機的市場宣傳，SPEC 將這些數字整理成一個單一數字，稱為 **Overall ssj_ops per watt**
- SPECpower_ssj2008 is basically a measure of **ssj_ops/watt** (server side Java operations per second per watt)
- Performance: ssj_ops/sec
- Power: Watts (Joules/sec)

$$\text{Overall ssj_ops per Watt} = \left(\sum_{i=0}^{10} \text{ssj_ops}_i \right) / \left(\sum_{i=0}^{10} \text{power}_i \right)$$

SPECpower_ssj2008 for Xeon X5650

Target Load %	Performance (ssj_ops)	Average Power (Watts)
100%	865,618	258
90%	786,688	242
80%	698,051	224
70%	607,826	204
60%	521,391	185
50%	436,757	170
40%	345,919	157
30%	262,071	146
20%	176,061	135
10%	86,784	121
0%	0	80
Overall Sum	4,787,166	1,922
$\Sigma \text{ssj_ops} / \Sigma \text{power} =$		2,490

實際應用中的測試程式

▣ 個人電腦跑分軟體(CPU/顯卡...)

- CINEBENCH

- Super PI

- 3D Mark

- AIDA 64

- CPU-Z

- ...



實際應用中的測試程式

▣ 行動裝置跑分App(智慧手機/平板)

- 安兔兔評測
- PCMark
- Geekbench(極客跑分)
- CPU-Z
- ...



1.10 謬誤與陷阱

- 在每章中均能找到的謬誤與陷阱這個小節的目的，是為了闡明一些你可能遇到的普遍的錯誤認知
 - 我們稱這些誤信為謬誤(fallacies)。當討論謬誤時，我們嘗試提出一個反例
 - 我們也談論陷阱(pitfalls)，即易犯的錯誤。陷阱常來自於將僅在限定的環境下成立的原則作延伸

1.10 謬誤與陷阱

- ▣ 陷阱：期望計算機某一方面的改善可以提昇整體效能到等同於該改善的幅度

- **Amdahl's 定律**

改善後的執行時間

$$= \frac{\text{受改善影響的執行時間}}{\text{改善的幅度}} + \text{不受影響的執行時間}$$

- ▣ (投資)回收遞減定律(The law of diminishing returns)

1.10 謬誤與陷阱

- ▣ 實情：整體效能提昇除了看項目改善的幅度還要看項目本身在整體的比重(比例原則)

- **Amdahl's 定律**

改善後的執行時間

$$= \frac{\text{受改善影響的執行時間}}{\text{改善的幅度}} + \text{不受影響的執行時間}$$

- ▣ (投資)回收遞減定律(The law of diminishing returns)

1.10 謬誤與陷阱

- ▣ 謬誤：低利用率的計算機幾乎不耗電
- ▣ 謬誤：為了效能而設計以及為了能量效益而設計是不相關的目標
- ▣ 陷阱：使用效能計算式的一部分作為效能衡量標準
 - **MIPS** (百萬指令每秒，million instructions per second)

$$\text{MIPS} = \frac{\text{指令數}}{\text{執行時間} \times 10^6}$$

- MIPS 的好處是其易懂，較快的計算機有較大的 MIPS，符合直覺

1.10 謬誤與陷阱

- ▣ 實情：低利用率的計算機因靜態功耗仍然會耗電！
- ▣ 實情：為了效能而設計以及為了能量效益而設計常常密切相關！Design for performance often closely relates to design for low power.
- ▣ MIPS比較僅對相同指令集架構的處理器有意義。

- **MIPS** (百萬指令每秒，million instructions per second)

$$\text{MIPS} = \frac{\text{指令數}}{\text{執行時間} \times 10^6}$$

- MIPS 的好處是其易懂，較快的計算機有較大的MIPS，符合直覺

Pitfall: MIPS as a Performance Metric

- MIPS: Millions of Instructions Per Second
 - Doesn't account for
 - Differences in ISAs between computers
 - Differences in complexity between instructions

$$\begin{aligned}\text{MIPS} &= \frac{\text{Instruction count}}{\text{Execution time} \times 10^6} \\ &= \frac{\text{Instruction count}}{\frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}} \times 10^6} = \frac{\text{Clock rate}}{\text{CPI} \times 10^6}\end{aligned}$$

- **CPI varies between programs on a given CPU**

1.10 謬誤與陷阱

- 有三個問題：

- ① MIPS說明指令執行速率然而未考慮指令的能力
- ② 即使在同一計算機上，MIPS 也隨程式而異
- ③ 程式可改寫成執行更多指令，然而各指令執行得更快！

1.11 總結評論

- ▣ 計算機設計者與程式師一定要對廣泛的不同議題有所瞭解
- ▣ 最重要的一個抽象方法的例證也許就是硬體與低階軟體間稱為指令集架構(instruction set architecture)的介面
 - 維持指令集恆常不變——在不同的成本及效能下以執行相同的軟體
- ▣ 執行時間是唯一有效且無可懷疑的效能量度
- ▣ 功耗已取代晶片面積成為微處理器設計最關鍵的資源
 - 節省用電且同時努力提昇效能已驅使硬體工業走向多核微處理器設計
 - 也因此驅使軟體工業走向平行硬體上面的程式寫作